

A Dual-Loop Teacher-Anchored Residual Calibration Framework for Drift-Adaptive GKP Decoding

Abstract

Approximate Gottesman–Kitaev–Preskill (GKP) quantum error correction uses continuous modular syndromes to infer displacement errors, but finite-energy states and drifting operating conditions make a fixed correction law brittle. We study a dual-loop decoding framework that preserves a deterministic affine fast path, $\Delta_t = K_t s_t + b_t$, while using recent syndrome histograms to update the runtime parameters at a slower cadence. The design keeps the per-shot computation compatible with quantized fixed-point execution and makes the adaptation target explicit: learning or calibration acts only on a low-dimensional parameter surface, not on the full decoding decision.

A classical teacher estimator provides an interpretable anchor for the affine parameters, and a lightweight CNN predicts only a bounded residual calibration term, currently focused on the bias b_t . This differs from end-to-end neural decoders and from outer-code soft-information decoders: the learned module updates a physical-layer calibration surface, whereas the per-shot path remains a deterministic affine rule. The intended systems advantage is to reduce logical failures under drift while keeping the fast boundary close to a small fixed-point controller.

Under the locked four-scenario mock-backed software-HIL protocol, the teacher-anchored hybrid residual branch is the winner in all four frozen scenarios, with UKF as the runner-up throughout. Bounded ablation and multi-seed materials support only a descriptive reading of this result: histogram-delta features matter, the instability pattern repeats across seeds, and the tested lower-clip intervention is mixed and mostly harmful, so the current mechanism story remains non-causal. A separately labeled supplement-side statistical-calibration extension lane further shows that a simple non-neural estimator can be strong under the same affine runtime contract, but its accepted closure remains no-promotion and without a unique clean threshold. The strongest current claim is therefore not that one CNN architecture closes the decoding problem, but that drift-adaptive affine calibration is a credible and hardware-bounded organizing principle for GKP fast-path decoding.

Contents

1	Introduction	2
2	Summary of Contributions	4
2.1	Metric-level advantages	5
3	Brief Review of the GKP Code	5
3.1	Ideal and approximate code states	5
3.2	Syndrome measurement as modular displacement information	5
3.3	Local affine decoding and its limits	6
3.4	Logical failure criterion	6

4	Noise and Drift Model	6
4.1	Effective noise state	6
4.2	Noise sources represented by the model	7
4.3	Drift scenarios	7
5	Model Architecture	7
5.1	Fast loop: affine fixed-point decoder	7
5.2	Parameter mapping	8
5.3	Teacher estimators	8
5.4	CNN residual branch	9
5.5	Statistical calibration branch	9
5.6	Stage-and-commit runtime contract	9
6	Relationship to Existing Work	10
6.1	GKP and bosonic analog soft information	10
6.2	Adaptive priors and syndrome-statistics estimation	10
6.3	Learned low-latency QEC modules	10
6.4	Real-time and FPGA QEC decoders	10
6.5	Advantages relative to existing QEC decoder approaches	10
7	Experimental Setup	11
7.1	Software-HIL protocol	11
7.2	Scenarios and modes	11
8	Numerical Results	12
8.1	Four-scenario affine benchmark	12
8.2	Feature and teacher ablations	12
8.2.1	Supplement-side statistical calibration extension lane	12
8.2.2	Local sensitivity inside the statistical-calibration extension lane	13
8.2.3	Per-scenario best variants and the no-promotion gate	13
8.3	Mechanism probe for residual-b behavior	13
8.4	Unseen drift generalization	13
8.5	Oracle and wrapped-Gaussian baselines	13
8.6	Runtime, quantization, and fixed-point degradation	14
8.7	Embedded runtime and board-level validation	14
8.8	Follow-up routes that remain outside the accepted result layer	14
9	Discussion	16
10	Conclusion	17

1 Introduction

Bosonic codes protect quantum information by encoding a logical qubit into the Hilbert space of an oscillator. The GKP code is a central example: ideal code states form a periodic lattice in phase space, so small shift errors can be diagnosed through modular syndrome measurements and corrected by displacement operations [1, 2]. Unlike binary stabilizer syndromes, GKP syndromes

are analog. Their continuous values carry information not only about the likely displacement, but also about how close the correction is to a lattice decision boundary.

The practical difficulty is that physical GKP states are approximate finite-energy states. Their comb peaks have finite width and an envelope, and syndrome measurements are further affected by measurement inefficiency, noisy auxiliary states, circuit-level imperfections, oscillator loss, and calibration error [2, 9]. As a result, the effective noise model seen by the decoder can drift over time. A correction law tuned for one operating point may become systematically mismatched after bias, variance, or covariance changes.

Recent GKP and bosonic-decoding work makes clear that analog soft information is valuable. Surface-GKP and concatenated bosonic-QLDPC decoders use continuous syndrome information to set matching weights, belief-propagation messages, or effective outer-code priors [4, 5, 6, 7, 8]. In parallel, hardware-facing QEC studies show that learned modules are most useful when their latency role and integration boundary are explicit [15, 16], and calibration work emphasizes that decoder priors must remain aligned with measured device statistics [10, 11, 12, 13, 14]. These two lessons motivate the present formulation.

Our starting point is that drift adaptation should enter the GKP layer through a bounded runtime contract rather than a free-form per-shot decoder. The fast loop is intentionally simple:

$$\Delta_t = K_t s_t + b_t, \tag{1}$$

where s_t is the measured two-quadrature GKP syndrome and (K_t, b_t) are quantized runtime parameters. A slower loop observes recent syndrome histograms, estimates an effective noise state, and stages updated parameters into the runtime bank. The online correction path therefore remains a small affine operation, while nonstationary estimation is handled asynchronously through teacher-guided or calibration-driven updates.

This separation gives the method a specific systems role. Compared with a fixed affine decoder, the parameters can track drift. Compared with a full neural decoder, the per-shot computation remains smaller, more interpretable, and easier to validate in fixed-point form. Compared with outer-code soft-information decoders, the adaptation target is the physical-layer GKP displacement rule itself rather than a matching graph or LDPC message schedule. The learned branch is therefore not the real-time decoder; it is one possible slow-loop calibration module operating on a low-dimensional affine parameter surface.

This also positions the work relative to FPGA-QEC practice. Much of the real-time hardware literature focuses on decoding binary stabilizer syndromes for surface codes, repetition-code experiments, or quantum-LDPC memories using lookup tables, union-find variants, matching or greedy approximations, clustering, neural decoders, or message-passing hardware [18, 19, 20, 21, 22, 24, 25, 27]. Those results establish the standard for closed-loop latency and resource reporting, but they do not directly address a GKP physical correction layer whose measured syndrome is analog, modular, and shaped by finite-energy state preparation. The present architecture is therefore complementary: it provides a low-dimensional calibration layer that could sit before, inside, or alongside a larger stabilizer or LDPC decoding stack while keeping the fast boundary as a small fixed-point affine displacement.

The current note keeps the evidence hierarchy explicit. The locked frozen software-HIL benchmark is the main result layer. Feature and teacher ablations, six-seed mechanism materials, and the separately labeled statistical-calibration extension lane are supporting layers that help explain or contextualize the benchmark but do not replace it. Training/material regeneration, isolated true `.tflite` execution, and real-board gate or provenance artifacts remain layered boundary evidence rather than coequal performance results.

2 Summary of Contributions

The work can be organized around six bounded contributions.

1. A dual-loop affine calibration formulation for approximate GKP decoding. The per-shot correction path is formulated as a bounded affine estimator $\Delta_t = K_t s_t + b_t$. The adaptation problem is moved into a slower loop that estimates an effective noise state from recent syndrome statistics and updates only the runtime parameters consumed by the fast path. This ties the continuous-syndrome GKP picture to a contract that remains compatible with quantized matrix-vector execution.

2. A teacher-anchored residual mainline architecture. A classical teacher estimator produces a stable baseline $(K_t^{\text{teacher}}, b_t^{\text{teacher}})$, and the learned branch predicts only a bounded residual correction,

$$\delta b_t = f_\phi(H_{t-c+1:t}, \Delta H_t, z_t^{\text{teacher}}), \quad (2)$$

with committed parameters

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (3)$$

Learning is therefore used to recalibrate a low-dimensional control surface, not to replace GKP decoding end-to-end or to move a neural network onto the per-shot critical path.

3. A locked mainline evidence layer. The strongest accepted performance evidence remains the locked four-scenario mock-backed software-HIL benchmark that serves as the frozen reference comparison for the paper. Within that fixed set, the teacher-anchored hybrid residual branch is the winner in all four scenarios and UKF is the runner-up throughout. This is the mainline ranking result; it is not an expanded benchmark claim, not a deployment-closure result, and not a real-board result.

4. Conservative supporting interpretation layers. Feature/teacher ablations and six-seed mechanism materials refine how the result may be discussed, but only within a descriptive boundary. The accepted reading is that histogram-delta features matter, a seed-bounded instability pattern repeats across the six-seed pack, and the tested lower-clip intervention is mixed and mostly harmful. The paper therefore retains descriptive mechanism support while rejecting any simple causal-closure or teacher-necessity retelling.

5. A separately labeled statistical-calibration extension lane. The same affine runtime contract can also host a simple non-neural histogram-driven calibration rule. In a separately labeled supplement-side calibration-extension lane, that statistical-calibration path can be strong relative to the frozen anchors, which supports the calibration-contract story at the project level. However, the accepted closure remains extension-lane only, explicitly no-promotion, and without a unique clean threshold.

6. A layered runtime and deployment-boundary evidence chain. The repository now has one code-backed training/material regeneration pack, one clean CPU-only bounded rerun, one isolated current-host true `.tfllite` runtime confirmation for selected preserved artifacts, and one read-only real-board gate/regeneration/provenance pack on a host that still cannot enter board

execution. These are useful supporting boundaries for the manuscript, but they remain layered evidence rather than coequal benchmark results or completed deployment claims.

2.1 Metric-level advantages

For reviewer-facing comparison, the advantages should be stated by metric and evidence layer rather than by architecture alone. Table 1 separates what is already supported by the locked software-HIL stack from what remains architecture-level reasoning or future hardware measurement.

The table is intentionally layered. Its job is not to flatten all evidence into one success claim, but to show which advantages are already ranking-level software-HIL facts, which are extension-lane or supporting facts, and which remain hardware-facing targets.

3 Brief Review of the GKP Code

This section fixes the physical-layer notation used later in the manuscript. It does not claim a new exact GKP decoder. Its purpose is to explain why a bounded affine fast path is a useful local approximation once drift adaptation is pushed into a slower calibration loop.

3.1 Ideal and approximate code states

The square-lattice GKP code encodes a qubit into an oscillator by using periodic structure in the q and p quadratures [1]. In the convention used here, the lattice constant is

$$\lambda = \sqrt{2\pi}. \quad (4)$$

An ideal logical state may be represented heuristically as an infinite comb, for example

$$|\bar{0}\rangle \propto \sum_{n \in \mathbb{Z}} |n\lambda\rangle_q. \quad (5)$$

Ideal comb states are not physical because they require infinite energy. Approximate GKP states replace the infinitely sharp comb with finitely squeezed peaks and an envelope. Finite-energy structure is central to practical GKP QEC rather than a minor implementation detail [2]: it determines the intrinsic syndrome uncertainty and contributes an effective displacement-like noise.

3.2 Syndrome measurement as modular displacement information

Let the accumulated phase-space displacement error before correction be

$$e_t = \begin{bmatrix} e_{q,t} \\ e_{p,t} \end{bmatrix}. \quad (6)$$

An ideal GKP syndrome observes this displacement modulo the lattice:

$$s_t = e_t \bmod \lambda, \quad s_{q,t}, s_{p,t} \in [-\lambda/2, \lambda/2). \quad (7)$$

In a finite-energy and noisy-measurement setting, the measured syndrome is more accurately described as

$$\tilde{s}_t = \text{mod}(e_t, \lambda) + \eta_t^{\text{meas}} + \eta_t^{\text{GKP}}. \quad (8)$$

Here η_t^{meas} represents measurement and auxiliary-state contributions, while η_t^{GKP} summarizes finite-squeezing uncertainty. The decoder therefore observes a noisy modular representative, not the absolute displacement.

3.3 Local affine decoding and its limits

The modulo structure makes exact GKP decoding nonlinear and branch dependent. Near a single lattice branch, however, a Gaussian approximation gives a useful local model. If e and s are jointly Gaussian,

$$\begin{bmatrix} e \\ s \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_e \\ \mu_s \end{bmatrix}, \begin{bmatrix} \Sigma_{ee} & \Sigma_{es} \\ \Sigma_{se} & \Sigma_{ss} \end{bmatrix} \right), \quad (9)$$

then the linear-MMSE estimate is

$$\hat{e} = \mu_e + \Sigma_{es} \Sigma_{ss}^{-1} (s - \mu_s) = Ks + b, \quad (10)$$

where

$$K = \Sigma_{es} \Sigma_{ss}^{-1}, \quad b = \mu_e - K\mu_s. \quad (11)$$

This derivation explains why an affine fast path is a plausible low-latency approximation. It also marks the boundary of the approximation. Near lattice decision boundaries, the posterior can be multimodal; maximum-likelihood, closest-lattice-point, or wrapped-Gaussian decoders can outperform a single global affine rule [3, 9]. The present framework keeps this affine approximation on the fast path and adapts it slowly through calibration of K and b , rather than claiming to solve the full wrapped posterior per shot.

3.4 Logical failure criterion

After correction, the residual displacement is wrapped into the GKP fundamental cell. A logical error occurs when the residual crosses a logical decision boundary, for example

$$|r_{q,t}| > \lambda/2 \quad \Rightarrow \quad X_L \text{ error}, \quad (12)$$

and similarly for the p -quadrature. Closed-loop logical error probability is therefore the central metric; parameter-regression error is useful only when it correlates with the downstream correction outcome. The mainline benchmark therefore evaluates logical performance under the locked software-HIL protocol, while runtime timing and hardware resource questions remain separate boundary issues.

4 Noise and Drift Model

The model below is an effective calibration model. Its role is to summarize the part of the analog GKP correction problem that the slow loop can estimate from recent syndrome statistics and map back into bounded affine runtime parameters. It is not a full circuit-level or hardware-validated noise closure.

4.1 Effective noise state

The slow loop does not attempt to infer every microscopic hardware parameter. It uses a low-dimensional effective state,

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t), \quad (13)$$

where σ_t controls the displacement scale, $\mu_{q,t}$ and $\mu_{p,t}$ represent mean bias, and ϑ_t captures covariance-axis rotation. This compact parameterization can be mapped to affine runtime parameters and estimated from finite histogram windows.

4.2 Noise sources represented by the model

The model is an effective displacement-noise model with additional measurement uncertainty. It absorbs several physical effects:

1. finite-energy GKP peak width and envelope effects;
2. Gaussian random displacement noise in q and p ;
3. biased displacement means caused by calibration offsets;
4. anisotropic or rotated covariance caused by asymmetric noise;
5. syndrome measurement noise and noisy auxiliary-state contributions.

This is narrower than a full circuit-level GKP noise model, but it is aligned with the affine fast-path contract. The compression is deliberate: the slow loop needs a control-oriented state that can be updated reliably from bounded windows, not an exhaustive microscopic device description.

4.3 Drift scenarios

The benchmark suite uses four drift scenarios:

1. `static_bias_theta`: a biased and rotated operating point;
2. `linear_ramp`: slowly changing effective noise parameters;
3. `step_sigma_theta`: a sudden variance/orientation change;
4. `periodic_drift`: periodic nonstationarity.

These scenarios separate different adaptation demands: steady calibration, tracking, shock response, and periodic following. They are sufficient for the current frozen benchmark and extension-lane follow-ups, but they are not claimed as exhaustive drift coverage.

5 Model Architecture

The architecture is organized around one mainline teacher-anchored residual path and several supporting or boundary layers that reuse the same affine runtime contract. Only the teacher-anchored hybrid path participates in the mainline frozen benchmark ranking; the statistical-calibration branch is kept as a separately labeled extension lane, and deployment-facing paths remain boundary evidence rather than completed hardware execution.

5.1 Fast loop: affine fixed-point decoder

The fast loop receives the current measured syndrome s_t , reads the active parameter bank, and computes

$$s_t^{\text{clip}} = \text{clip}(s_t, -s_{\text{max}}, s_{\text{max}}), \quad (14)$$

$$\Delta_t^{\text{raw}} = K_t s_t^{\text{clip}} + b_t, \quad (15)$$

$$\Delta_t = Q(\text{clip}(\Delta_t^{\text{raw}}, -\Delta_{\text{max}}, \Delta_{\text{max}})), \quad (16)$$

where $Q(\cdot)$ denotes fixed-point quantization. The implementation uses an FPGA-oriented Q4.20 representation for syndrome values and runtime parameters. The fast loop also records diagnostic counters, including histogram-input saturation, correction saturation, overflow, aggressive parameter events, commit counts, and fallback-related signals. The current hardware-bounded claim attaches to this fast boundary and its runtime contract, not to completed board-level timing or resource closure.

5.2 Parameter mapping

Given an estimated noise state, the parameter mapper constructs an error covariance

$$C = R(\vartheta) \begin{bmatrix} \sigma_q^2 & 0 \\ 0 & \sigma_p^2 \end{bmatrix} R(\vartheta)^\top, \quad (17)$$

and a measurement covariance

$$R_{\text{meas}} = (\sigma_{\text{meas}}^2 + \Delta_{\text{eff}}^2)I. \quad (18)$$

The raw gain is

$$K_{\text{raw}} = C(C + R_{\text{meas}})^{-1}, \quad (19)$$

followed by gain clipping or scaling. The bias target is

$$b_{\text{target}} = \alpha(I - K_{\text{target}})\mu, \quad \mu = \begin{bmatrix} \mu_q \\ \mu_p \end{bmatrix}. \quad (20)$$

Finally, exponential smoothing produces staged parameters:

$$K_t = (1 - \beta)K_{t-1} + \beta K_{\text{target}}, \quad b_t = (1 - \beta)b_{t-1} + \beta b_{\text{target}}. \quad (21)$$

This mapper defines the common committed interface shared by classical baselines, the teacher-anchored learned residual branch, and the bounded statistical-calibration extension lane.

5.3 Teacher estimators

The teacher family provides stable estimates of θ_t^{noise} from recent syndrome history. A simple teacher computes moments over a histogram window. Stronger teachers such as EKF, UKF, RLS, or particle-filter variants add state-space assumptions over time. Abstractly,

$$\hat{\theta}_t^{\text{teacher}} = \text{Teacher}(H_{1:t}), \quad (22)$$

and

$$(K_t^{\text{teacher}}, b_t^{\text{teacher}}) = \text{ParamMapper}(\hat{\theta}_t^{\text{teacher}}). \quad (23)$$

The teacher keeps the learned branch near an interpretable calibration surface and creates meaningful ablation baselines. In the locked mainline comparison, these teacher-driven baselines define the classical reference rows against which the hybrid residual branch is judged.

5.4 CNN residual branch

The CNN consumes a short context of normalized syndrome histograms and histogram deltas:

$$X_t^{\text{hist}} = [H_{t-c+1}, \dots, H_t, \Delta H_{t-c+2}, \dots, \Delta H_t]. \quad (24)$$

Teacher-side features may include

$$z_t^{\text{teacher}} = (\hat{\theta}_t^{\text{teacher}}, K_t^{\text{teacher}}, b_t^{\text{teacher}}, \Delta b_t^{\text{teacher}}, \dots). \quad (25)$$

The gated branch narrows this to a small set of teacher- b and teacher- Δb scalars. The learned branch predicts

$$\hat{\delta b}_t = f_\phi(X_t^{\text{hist}}, z_t^{\text{teacher}}), \quad (26)$$

then clips the result:

$$\delta b_t = \text{clip}(s_b \hat{\delta b}_t, -b_{\max}, b_{\max}). \quad (27)$$

The committed parameter is

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (28)$$

This matches the strongest current mainline evidence: the accepted learned winner is a teacher-anchored residual-b style branch under a bounded affine runtime contract, not an unconstrained neural decoder that replaces the fast path.

5.5 Statistical calibration branch

The same affine runtime contract also supports a non-neural statistical calibration branch. This branch estimates a bounded bias correction directly from recent syndrome statistics and emits runtime parameters through the same clipping, smoothing, and stage-and-commit path. Its role is both practical and scientific: it tests whether the benefit comes from a general histogram-driven calibration principle rather than from the CNN architecture alone. However, its accepted status in the current manuscript remains narrow: it is a separately labeled FR8 extension lane, not part of the frozen ‘T24’ mainline ranking, not a promoted comparator, and not a unique-threshold closure.

5.6 Stage-and-commit runtime contract

The slow loop does not directly mutate active fast-loop parameters. It stages a candidate (K, b) into an inactive bank and commits it at a safe epoch boundary:

$$(K_t, b_t) = \begin{cases} (K^A, b^A), & t < t_{\text{commit}}, \\ (K^B, b^B), & t \geq t_{\text{commit}}. \end{cases} \quad (29)$$

This contract allows stale-parameter effects, update latency, commit success, rollback/fallback behavior, and fixed-point stability to be measured separately from logical performance. Software-HIL validates that this runtime contract can be exercised and audited, while current real-board materials stop at a read-only gate/regeneration/provenance pack with present-host ‘NO_GO’.

6 Relationship to Existing Work

6.1 GKP and bosonic analog soft information

GKP analog syndrome information is already known to be useful. Analog-QEC, surface-GKP, and concatenated bosonic-QLDPC decoders use continuous measurement information to set decoder weights or soft messages [3, 4, 5, 6, 7, 8]. The distinction here is not the value of analog information itself, but the layer at which it is consumed: recent syndrome statistics are compressed into physical-layer affine GKP fast-path parameters rather than outer-code matching weights or LDPC messages.

6.2 Adaptive priors and syndrome-statistics estimation

Adaptive-weight and syndrome-statistics methods estimate decoder weights or noise parameters from measured data [10, 11, 12]. Calibrated decoders and decoder-prior optimization show that prior mismatch can materially affect logical performance [13, 14]. This work takes the same lesson seriously, but maps the adaptation target to the GKP affine parameters (K, b) and constrains the update through an explicit stage-and-commit runtime contract.

6.3 Learned low-latency QEC modules

AI pre-decoders, high-accuracy neural decoders, and calibration-conditioned FiLM decoders demonstrate a systems-oriented pattern: a learned module is most useful when its input/output contract, latency role, and integration with a classical decoder are explicit [15, 16, 17]. The learned module here is narrower. It is not the per-shot decoder, but a slow calibration component that updates low-dimensional affine parameters while the fast boundary remains fixed-point affine logic.

6.4 Real-time and FPGA QEC decoders

Real-time QEC decoders and FPGA-integrated systems have reported low-latency hardware decoding in stabilizer-code settings [24, 25, 26, 27]. Hardware-oriented decoder designs also include lookup-table approaches, union-find architectures, matching and greedy approximations, clustering decoders, and FPGA-tailored quantum-LDPC message-passing systems [18, 19, 20, 21, 22, 23]. Real-time bosonic QEC has also been demonstrated in experimental feedback systems [28, 29]. These works set the standard for full deployment claims: closed-loop timing, worst-case latency, resource use, and hardware integration.

The contribution here is intentionally narrower and complementary. It does not aim to replace surface-code, repetition-code, or QLDPC hardware decoders. Instead, it targets the GKP physical-layer correction problem that precedes many concatenated-code decoding tasks: how should a device update the analog displacement rule when the finite-energy GKP syndrome distribution drifts? In that setting, a fixed binary graph decoder is not the natural first abstraction, and an unconstrained neural network on the per-shot path is not a compelling hardware endpoint. A staged affine parameter bank provides a smaller and more testable contract: update (K, b) slowly from recent analog statistics, then execute only $Ks + b$ at the fast boundary.

6.5 Advantages relative to existing QEC decoder approaches

The proposed architecture is best viewed as a middle layer between physical GKP syndrome acquisition and higher-level QEC decoding. Its advantages over nearby approaches are architectural and evidence-bounded rather than claims of completed board-level deployment or promoted comparator closure.

Preserving analog GKP information at the right layer. Outer-code GKP methods already show that analog syndrome values improve matching weights or belief-propagation messages. This work uses the same principle earlier in the stack: the analog histogram updates the physical GKP displacement rule itself. This can reduce prior mismatch before the information is handed to a surface-code, LDPC, or other outer decoder.

Drift adaptation without a neural real-time critical path. End-to-end neural decoders can be expressive, but they make worst-case latency, fixed-point validation, and fallback behavior harder to reason about. Here, learning is optional and narrow: a CNN residual branch or, in a separately bounded extension lane, a statistical calibration branch updates a small parameter surface, while the real-time path remains deterministic affine logic. This separation is expected to be useful when future hardware systems need both adaptation and bounded per-shot timing.

A replaceable calibration module rather than a single decoder commitment. The same runtime contract can host a teacher estimator, a learned residual branch, or a non-neural statistical calibration rule. The bounded statistical-calibration extension lane therefore supports the design story without changing the accepted hierarchy: the important object is the histogram-driven affine calibration contract, not one particular CNN architecture, and this note keeps that lane supplement-side rather than as a promoted mainline comparator.

Explicit deployment diagnostics. The framework makes quantization, clipping, saturation, stale-parameter use, commit counts, fallback behavior, and update latency visible benchmark objects. This is closer to FPGA-QEC practice than reporting only offline regression error, and it provides a path to future resource and timing measurements once a real hardware host is available.

Compatibility with larger QEC stacks. Because the fast path emits a physical displacement correction rather than a complete outer-code decision, the method can in principle be combined with surface-GKP, bosonic-QLDPC, or conventional stabilizer-code decoders. Its expected role is to keep the GKP layer calibrated under drift so that downstream decoders receive better-conditioned residual information.

7 Experimental Setup

7.1 Software-HIL protocol

The mainline numerical experiments use a locked mock-backed software-HIL protocol that preserves the fast loop, slow loop, parameter-bank, and stage-and-commit interfaces while executing the underlying experiment in software. The authoritative frozen reference benchmark contains four drift scenarios, five mainline modes, paired seeds, and two repeats under one fixed comparison contract. This protocol is sufficient for comparing adaptive calibration laws under matched drift trajectories and shared random structure, but it does not measure board-level latency, hardware resource use, or deployment closure. Later ablation, multi-seed, and calibration-extension materials are therefore interpreted relative to this anchor rather than as a rewritten benchmark.

7.2 Scenarios and modes

The four drift scenarios are:

Scenario key	Interpretation
<code>static_bias_theta</code>	biased and rotated operating point
<code>linear_ramp</code>	slowly changing effective noise parameters
<code>step_sigma_theta</code>	sudden variance/orientation change
<code>periodic_drift</code>	periodic nonstationarity

The mainline comparison includes five modes: EKF, UKF, a constant-mean affine baseline, an RLS residual-*b* baseline, and the learned hybrid residual branch. Additional bounded materials then cover feature and teacher ablations, six-seed descriptive mechanism snapshots, and a separately labeled statistical-calibration extension lane under the same affine runtime contract. The reported metric is `final_ler_mean`, where lower values indicate fewer logical failures under the fixed software-HIL protocol.

8 Numerical Results

This section separates the locked benchmark result from supporting interpretation layers and from follow-up routes that remain explicitly outside the accepted result layer. In the current frozen-mainline handoff, Table 2 remains the primary in-text representation of the locked benchmark layer, while appendix, supplement, and excluded surfaces remain routed separately.

8.1 Four-scenario affine benchmark

Table 2 records the locked frozen benchmark that also underlies any later paper-facing visual summary. In this note, the table is kept as the primary representation because it is the authoritative numeric surface and the least ambiguous freeze point. Under the locked frozen reference protocol, the hybrid residual branch is the winner in all four frozen scenarios, with UKF as the runner-up throughout. This remains the authoritative mainline result layer of the current note. It supports a bounded mock-backed software-HIL ranking only, and it should not be retold as an expanded benchmark, a `.tflite` ranking, or a board-level ranking.

8.2 Feature and teacher ablations

Table 3 is best read as appendix-side support for the frozen benchmark rather than as a replacement ranking table. Removing histogram deltas degrades performance relative to the full hybrid model, indicating that temporal changes in the syndrome histogram matter inside this fixed benchmark set. Removing teacher-side channels also changes performance materially, but the no-teacher-params row performing best means the current evidence does not support a simple teacher-necessity claim. The safest reading is therefore descriptive: feature and teacher design matter, yet the bounded ablation does not close the mechanism question.

The next three lower-level note items preserve the accepted statistical-calibration evidence as a supplement-side extension lane rather than as a coequal Results pillar.

8.2.1 Supplement-side statistical calibration extension lane

A separately labeled extension lane adds statistical calibration variants under the same affine runtime contract. Table 4 records the bounded comparison snapshot that motivated this lane: within that bounded comparison matrix, statistical calibration variants outperform UKF and the learned hybrid branch. However, the accepted closure deliberately does not promote this lane into the mainline comparator set. The locked five-mode frozen benchmark remains the authoritative mainline

result, and the statistical-calibration lane is retained as supplement-side evidence for the broader adaptive-affine contract.

8.2.2 Local sensitivity inside the statistical-calibration extension lane

The local grid around the default setting shows that the extension-lane signal is not tied to a single pathological point, but it also does not identify a unique clean threshold choice. The high-threshold variant has the best aggregate mean LER by a negligible margin, whereas the default variant wins three of four scenarios and has the best mean within-lane rank. This persistent tie is why the accepted closure remains no-promotion rather than collapsing to one promoted statistical-calibration setting.

8.2.3 Per-scenario best variants and the no-promotion gate

Table 6 reports the per-scenario winners inside the extension lane. These rows show why the lane remains scientifically interesting, but they are not sufficient to override the no-promotion gate: the default and high-threshold settings remain persistently tied across aggregate and scenario-wise summaries, and the `static_bias_theta` winner also carries a mixed-status caveat. The safest use of these numbers is therefore supplement-side documentation of the lane, not a claim that a mature promoted comparator has been established.

8.3 Mechanism probe for residual-b behavior

A paper-facing mechanism figure may summarize the learned residual branch, but appendix tables remain the authoritative numeric companion. The six-seed evidence is descriptive rather than causal: the instability pattern repeats broadly, while the tested lower-clip intervention is mixed and mostly harmful. This supports a cautious interpretation: residual amplitude participates in some failure modes, but it is not a monotonic causal explanation by itself, and the current evidence does not prove teacher necessity. Appendix tables carry the bounded ablation, cross-seed snapshots, and provenance that support this conservative reading without reopening the main-text route.

8.4 Unseen drift generalization

The current four scenarios cover steady bias, ramp tracking, step response, and periodic drift. A stronger benchmark would require unseen drift families such as random-walk drift, $1/f$ -like drift, burst measurement noise, coupled bias-variance drift, and faster-than-window drift. That expansion remains a future benchmark lane rather than a completed claim in the present note; it is precisely the kind of follow-up needed to distinguish genuine drift adaptation from overfitting to a small set of hand-designed trajectories.

8.5 Oracle and wrapped-Gaussian baselines

The affine methods still need comparison against stronger theoretical baselines: nearest-lattice hard decoding, static affine decoding, oracle affine decoding with true noise state, and wrapped-Gaussian or maximum-likelihood decoding in small settings. Those baselines would separate two kinds of loss that remain entangled here: the loss from constraining the fast path to be affine, and the loss from estimating the noise state imperfectly in the slow loop.

8.6 Runtime, quantization, and fixed-point degradation

The deployment story depends on more than logical performance, so the current manuscript keeps runtime material at a supporting-boundary layer rather than flattening it into a single deployment claim. The strongest accepted runtime-facing fact is narrow but real: selected preserved float and int8 `.tflite` artifacts have been loaded and executed in one isolated current-host environment, and that fact is strong enough to justify an appendix-level runtime boundary table for those preserved artifacts. What remains unclosed is the broader fixed-point/runtime degradation question itself. Saturation and overflow rates under deployment-style quantization, commit latency, stale-parameter penalty, fallback frequency, host-side update cost, and source-vs-embedded drift under a default environment still require separate bounded tasks rather than prose promotion.

8.7 Embedded runtime and board-level validation

Embedded runtime and board-level validation must remain split across two weaker layers. The first is an appendix-level current-host runtime fact: selected preserved `.tflite` artifacts run in one isolated environment, which is enough to discuss true runtime existence for those artifacts but not enough to claim default-environment compatibility, software-HIL closure, or deployment closure. The second is a supplement-only hardware gate fact: the checked-in read-only real-board gate/regeneration/provenance lane still ends in `NO_GO_REAL_BOARD_HOST_OR_DEVICE_PATH_UNAVAILABLE`. Because the repository currently lacks a usable `Linux + FPGA` host, openable device paths, and measured board timing/resource rows, hardware-dependent execution surfaces remain blocked rather than merely postponed prose.

8.8 Follow-up routes that remain outside the accepted result layer

The main benchmark and paper-material lane should remain separated from parallel extension experiments. This subsection is therefore a reader-facing boundary register rather than an additional Results pillar: it records follow-up routes that may be explored in parallel with the mainline, but whose outputs remain sidecar evidence until a later promotion gate decides otherwise. In particular, these routes must not rewrite the frozen four-scenario anchor, promote the statistical-calibration lane into a mature comparator, or turn software-HIL evidence into `.tflite` or board-level validation.

Wave A lists the immediately bounded sidecar lanes. These lanes preserve the fast-loop contract and mostly operate through cached replay, toy simulation, or deterministic contract tests.

Wave A lane	Experimental question	First safe execution
Temporal histogram TCN	Does a short causal sequence of histogram windows, deltas, and compact statistics improve residual- b or drift-regime prediction beyond the current short-window features?	Cached replay or tiny TCN residual-head probe under a bounded sidecar run root.
Adaptive teacher confidence	Can a syndrome-only or teacher-output replay estimate confidence, freeze unsafe updates, or fall back to a conservative anchor before a bad parameter commit?	Offline teacher-policy replay with explicit confidence and fallback fields.
Gain-scheduled parameter bank	Can a finite piecewise-affine bank reduce parameter jitter or improve drift tracking while keeping the fast path as $\Delta = K_i s + b_i$?	Bank-selection toy replay with range, thrashing, and switch-frequency diagnostics.
Atomic commit and rollback	Which stage, commit, acknowledgement, freeze, rollback, and timeout states are needed before stronger runtime integration is credible?	Deterministic mock contract tests only; no MMIO, DMA, or board writes.

Wave B lists the second-wave candidate pool. These routes are useful for making the paper experimentally complete, but they should start as S0 design or toy probes rather than as long formal benchmarks.

Wave B lane	Experimental question	Boundary
Moments and EWMA feature probes	Do low-cost summaries such as marginals, mean/variance drift, EWMA, entropy, anisotropy, or edge-mass features explain the same gains as the full histogram image?	Feature ablation only; not a new formal benchmark by itself.
Teacher-conditioned residual head	Does conditioning the residual head on teacher state, teacher deltas, or confidence improve stability without making the learned module the per-shot decoder?	Small retrain or replay; still a bounded slow-loop calibration module.
State-space or lightweight sequence model	Are modern compressed temporal models such as tiny state-space, Mamba-like, or recurrent/transformer toy heads useful after the TCN baseline is known?	Research-first toy route; no full decoder replacement and no fast-loop change.
Outer-code soft-information positioning	How would the physical GKP calibration layer interface with surface-GKP, GKP-surface, or QLDPC-GKP soft-information decoders?	Positioning memo or tiny interface toy; not a current mainline code-family benchmark.

The purpose of this wave structure is not to multiply unrelated experiments. It keeps the scientific question fixed: the fast path remains a deterministic affine correction, while sidecar lanes ask which slow-loop estimator, feature summary, teacher policy, or commit policy most safely updates the affine parameters under drift. Negative or inconclusive sidecar results are also useful, because they can show that the current histogram-conditioned affine contract is already near the useful complexity boundary.

9 Discussion

The results and architecture suggest a useful reframing of the project. The central claim should not be that a CNN alone is a universally superior GKP decoder. A more defensible claim is that drift-adaptive GKP correction can be organized as a low-dimensional affine calibration problem with a deterministic fast path and replaceable slow calibration modules. Under this framing, the teacher-anchored CNN branch, the statistical calibration branch, and the filtering baselines are all instances of the same runtime question: how should recent syndrome statistics update (K, b) without disturbing the per-shot latency contract?

This framing clarifies why the method is different from several neighboring approaches. It is not a conventional FPGA surface-code decoder, because it does not operate on a binary detection-event graph. It is not an outer-code GKP soft-information decoder, because it updates the physical displacement rule before outer-code message passing or matching. It is not an end-to-end neural QEC decoder, because the neural component is not placed on the per-shot critical path. It is also not simply a calibration pre-processing routine, because the estimated state is committed through the same quantized, clipped, double-buffered runtime interface that the fast loop consumes.

The advantage should be interpreted at two levels. At the mainline project level, the strongest accepted result is still the locked four-scenario software-HIL ranking: the teacher-anchored hybrid branch beats the filtering baselines across all frozen scenarios. At the model-comparison level, the current evidence does not require claiming that the CNN is always the best estimator. A separately labeled calibration-extension supplement is instead useful because it shows that the same runtime contract can host a strong non-neural alternative; however, the accepted closure still keeps that lane supplement-side and no-promotion because the default and high-threshold settings remain persistently tied. The learned branch therefore remains valuable as a replaceable slow-loop module rather than as a uniquely validated final comparator.

The expected long-term advantage is therefore modularity under real-time constraints. If a future device requires a stronger statistical estimator, a learned temporal model, or a more conservative fallback policy, the slow calibration module can be replaced without changing the affine hardware boundary. Conversely, if future GKP experiments require tighter latency, the fast path can remain a fixed-point matrix-vector operation while the slow estimator is improved offline or on a host processor. This is the main systems-level reason to prefer the proposed contract over a monolithic decoder design: the architecture can trade estimator sophistication against update cadence without putting neural inference or a graph decoder on the per-shot critical path.

The boundary of the current evidence remains important, and it is now clearer to organize it as a layered manuscript route rather than a single deployment story. Main-text claims can stop at the frozen-set ranking and its conservative mechanism/extension-lane interpretation. Appendix-level supporting materials can carry the canonical training/material provenance table and the isolated current-host true `.tflite` runtime table for selected preserved artifacts. Supplement-only materials can carry the calibration-extension supplement with its no-promotion / no-unique-threshold gate and the read-only real-board gate/regeneration/provenance pack. What remains blocked is any hardware-dependent surface that needs a future `Linux + FPGA` host, openable device paths, board-level latency/resource measurements, or deployment-grade portability. Keeping these layers separate is not a weakness of the write-up; it is the mechanism that prevents supporting evidence from being misread as new ranking evidence or completed hardware validation. At this point, the mainline note is best kept frozen at a bounded manual-finish state rather than reopened for a new evidence layer. The frozen route keeps the ranking and conservative mechanism reading in the main text, places bounded ablation plus training/material provenance and runtime-boundary tables in the appendix, retains the calibration-extension and real-board gate materials in the supplement,

and leaves blocked hardware or portability surfaces outside the package claims.

10 Conclusion

This work frames drift-adaptive GKP correction as a dual-loop affine calibration problem. The fast loop remains a bounded fixed-point rule, $\Delta_t = K_t s_t + b_t$, while a slower loop updates the affine parameters from recent syndrome statistics. This formulation creates a practical middle ground between fixed decoders, which cannot track hardware drift, and full neural decoders, which can be difficult to place on a deterministic per-shot path.

The numerical results support two main conclusions. First, under the locked four-scenario software-HIL suite, adaptive affine decoding with a teacher-anchored residual branch improves over filtering baselines, while bounded ablations and multi-seed mechanism materials show that histogram-delta features carry useful information and that teacher-side inputs do not admit a simple necessity story. Second, a separately labeled supplement-side calibration-extension lane shows that simple non-neural estimators can also be strong under the same runtime contract, but the accepted closure remains no-promotion and no unique clean threshold. Together, these results suggest that the strongest scientific story is not that one CNN architecture has closed the comparison, but that low-dimensional histogram-driven calibration of an affine GKP fast path is a promising structure for drift-adaptive decoding.

The remaining technical gap is therefore layered, not singular. On the manuscript side, this note is now frozen around a simulation/material-first route: frozen-set main results in the main text, bounded ablation plus training/material provenance and isolated true runtime tables in the appendix, and a calibration-extension supplement plus real-board gate/provenance materials in the supplement. That frozen route records the bounded manual finish already executed here rather than reopening another round of assembly or claim upgrade. On the execution side, broader drift families, stronger oracle or wrapped-Gaussian baselines, default-environment `.tflite` portability, and any board-level timing claim still require separate bounded tasks. In particular, because there is no current `Linux + FPGA` host, the hardware-dependent surface remains blocked rather than silently delayed. The honest handoff at this stage is therefore a frozen mainline note with explicit blocked-surface disclaimers, not deployment closure, submission-ready completion, or hardware-ready finalization. That handoff may package main-text results, appendix support, and supplement-only gated surfaces together, but it must continue to exclude blocked hardware, portability, and promoted-comparator claims.

References

- [1] D. Gottesman, A. Kitaev, and J. Preskill, “Encoding a qubit in an oscillator,” *Physical Review A*, 2001.
- [2] A. L. Grimsmo and S. Puri, “Quantum Error Correction with the Gottesman–Kitaev–Preskill Code,” *PRX Quantum* 2, 020101, 2021.
- [3] K. Fukui, A. Tomita, A. Okamoto, and K. Fujii, “High-threshold fault-tolerant quantum computation with analog quantum error correction,” *Physical Review X*, 2018.
- [4] K. Noh and C. Chamberland, “Fault-tolerant bosonic quantum error correction with the surface-GKP code,” *Physical Review A*, 2020.

- [5] K. Noh, C. Chamberland, and F. G. S. L. Brandao, “Low-overhead fault-tolerant quantum error correction with the surface-GKP code,” *PRX Quantum*, 2022.
- [6] N. Raveendran, N. Rengaswamy, F. Rozpedek, A. Raina, L. Jiang, and B. Vasic, “Finite-rate QLDPC-GKP coding scheme that surpasses the CSS Hamming bound,” *Quantum*, 2022.
- [7] L. Berent et al., “Analog information decoding of bosonic quantum LDPC codes,” *PRX Quantum*, 2024.
- [8] S. K. Borah, A. K. Pradhan, N. Raveendran, M. Pacenti, and B. Vasic, “Fault tolerant decoding of QLDPC-GKP codes with circuit level soft information,” preprint, 2025.
- [9] M.-A. Roy, T. Pousset, and B. Royer, “Decoding multimode Gottesman–Kitaev–Preskill codes with noisy auxiliary states,” preprint, 2025.
- [10] S. Spitz, B. Tarasinski, C. W. J. Beenakker, and T. E. O’Brien, “Adaptive weight estimator for quantum error correction,” *Advanced Quantum Technologies*, 2018.
- [11] T. Wagner, H. Kampermann, and D. Bruß, “Optimal noise estimation from syndrome statistics of quantum codes,” *Physical Review Research*, 2021.
- [12] H. Chen et al., “DGR: Tackling drifted and correlated noise in quantum error correction via decoding graph re-weighting,” preprint, 2023.
- [13] E. H. Chen et al., “Calibrated decoders for experimental quantum error correction,” *Physical Review Letters*, 2022.
- [14] V. Sivak, M. Newman, and P. Klimov, “Optimization of decoder priors for accurate quantum error correction,” *Physical Review Letters* 133, 150603, 2024.
- [15] C. Chamberland, J. Olle, M. Li, S. Thornton, and I. Baratta, “Fast and accurate AI-based pre-decoders for surface codes,” arXiv:2604.12841, 2026.
- [16] S. Stein, S. Kan, C. Liu, A. Harkness, S. Garner, Z. Du, Y. Ding, Y. Mao, and A. Li, “Calibration-conditioned FiLM decoders for low-latency decoding of quantum error correction evaluated on IBM repetition-code experiments,” preprint, 2026.
- [17] Google Quantum AI and collaborators, “Learning high-accuracy error decoding for quantum processors,” preprint, 2023.
- [18] “LILLIPUT: A lightweight low-latency lookup-table based decoder for near-term quantum error correction,” *ASPLOS*, 2022.
- [19] “AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers,” *HPCA*, 2022.
- [20] “Scalable quantum error correction for surface codes using FPGA,” preprint, 2023.
- [21] “Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching acceleration,” *ISCA*, 2023.
- [22] “A real-time, scalable, fast and resource-efficient decoder for a quantum computer,” *Nature Electronics*, 2025.

- [23] “FPGA-tailored algorithms for real-time decoding of quantum LDPC codes,” preprint, 2025.
- [24] L. Caune et al., “Demonstrating real-time and low-latency quantum error correction with superconducting qubits,” preprint, 2024.
- [25] X. Yang et al., “Real-time surface-code error correction using an FPGA-based neural-network decoder,” preprint, 2026.
- [26] M. Ziad et al., “Local clustering decoder: a fast and adaptive hardware decoder for the surface code,” preprint, 2024.
- [27] L. Maurer et al., “Real-time decoding of the gross code memory with FPGAs,” preprint, 2025.
- [28] V. Sivak et al., “Real-time quantum error correction beyond break-even,” *Nature*, 2023.
- [29] D. Lachance-Quirion et al., “Autonomous quantum error correction of Gottesman–Kitaev–Preskill states,” *Physical Review Letters*, 2024.

Metric	Advantage	Architectural reason	Evidence boundary
Logical-error proxy	Lower error on the locked frozen software-HIL benchmark than the five mainline baselines; the separately labeled statistical-calibration extension lane can also outperform the frozen anchors under its bounded protocol.	Both learned and statistical branches update the same physical-layer affine surface, so adaptation acts directly on the displacement rule used by the GKP fast loop.	Supported for the ‘T24’ mainline ranking and the FR8 extension-lane supplement; not a paper-grade expanded benchmark and not a promoted comparator.
Per-shot latency	Low latency by design on the correction path.	The fast loop evaluates only a 2×2 affine map, bias addition, clipping, quantization, and parameter-bank selection; CNN or statistical estimation is kept off the per-shot critical path.	Architectural claim; board timing and worst-case latency remain a hardware measurement target.
FPGA resource and cost	Expected small fast-path resource footprint and lower verification burden than placing a neural or graph decoder in the real-time path.	The real-time datapath needs only fixed-point multiply-adds, saturation logic, and a compact parameter bank; the slow estimator can run less frequently on a host, embedded processor, or offline update service.	Resource use and energy/cost are not yet measured on a real device.
Engineering implementation	The control boundary is simple to integrate, test, and fail safely.	Quantized (K, b) , clipping limits, stale-parameter handling, saturation counters, and double-buffered stage-and-commit updates form explicit runtime contracts.	Software-HIL validates the interface shape; the real-board lane remains a read-only gate/provenance boundary with current-host ‘NO_GO’.
Drift robustness	Adaptation can track mean, variance, and orientation changes in the effective GKP noise state.	Histogram windows and teacher/statistical summaries update the affine parameters over time instead of freezing one calibration point.	Supported for the current four-scenario effective-drift suite; broader drift families and holdout protocols remain benchmark-expansion targets.
Modularity	The best slow-loop estimator can change without changing the fast path.	Teacher residual CNN, statistical calibration, temporal models, and gain-scheduled parameter banks all map to the same committed (K, b) interface.	Current evidence supports this mainly through the teacher-anchored hybrid path and the supplement-side statistical-calibration extension lane; other slow-loop variants remain future-work candidates.
Compatibility with larger QEC stacks	The method can serve as a GKP physical-layer calibration module before an outer stabilizer, surface-GKP, or LDPC-GKP decoder.	It improves the physical displacement correction and residual analog statistics instead of replacing the outer-code decoder decision rule.	System-level concatenated-code gains remain future-work evidence.

Table 1: Metric-level advantage summary. The strongest current evidence is logical-error reduction in software-HIL, with ‘statcalib’ retained only as a separately labeled extension-lane supplement. Latency, resource, and cost advantages are architecture-level consequences of the affine fast path and should be measured on hardware before being written as deployment results.

Scenario	EKF	UKF	Const.- μ	RLS- b	Hybrid- b
<code>static_bias_theta</code>	0.838110	0.825370	0.836658	0.837577	0.810902
<code>linear_ramp</code>	0.819200	0.811201	0.816911	0.819373	0.787755
<code>step_sigma_theta</code>	0.822365	0.811548	0.819784	0.821493	0.788800
<code>periodic_drift</code>	0.832192	0.821558	0.829670	0.832334	0.806392

Table 2: Four-scenario affine benchmark. Entries are `final_ler_mean`; lower is better.

Mode	Avg. LER	Δ vs UKF	Δ vs Hybrid Full
<code>ukf</code>	0.817382	0.000000	+0.018837
<code>hybrid_full</code>	0.798545	-0.018837	0.000000
<code>hybrid_no_hist_deltas</code>	0.826723	+0.009341	+0.028178
<code>hybrid_no_teacher_prediction</code>	0.807251	-0.010131	+0.008706
<code>hybrid_no_teacher_params</code>	0.749621	-0.067761	-0.048924
<code>hybrid_no_teacher_deltas</code>	0.800329	-0.017053	+0.001784

Table 3: Feature and teacher ablation summary. Negative Δ means improvement over the reference.

Scenario	UKF	Hybrid- b	StatCalib
<code>static_bias_theta</code>	0.825370	0.810902	0.431708
<code>linear_ramp</code>	0.811201	0.787755	0.467083
<code>step_sigma_theta</code>	0.811548	0.788800	0.460016
<code>periodic_drift</code>	0.821558	0.806392	0.438751

Table 4: Bounded statistical-calibration extension-lane comparison under the same drift suite. Entries are `final_ler_mean`; lower is better. This table does not by itself promote statistical calibration into the mainline comparator set.

Variant	Mean LER	Mean Rank	Scenario Wins
<code>statcalib_high_threshold</code>	0.449241	1.75	1
<code>statcalib_default</code>	0.449254	1.25	3
<code>statcalib_high_scale</code>	0.456477	3.00	0
<code>statcalib_low_clip</code>	0.481484	4.50	0
<code>statcalib_low_scale</code>	0.485129	4.50	0

Table 5: Local sensitivity within the bounded statistical-calibration extension lane. Mean rank is computed within the five variants for each scenario; the near-tie between default and high-threshold prevents a unique promoted setting.

Scenario	Best variant	Best LER	Gap vs Hybrid- b
<code>static_bias_theta</code>	<code>high_threshold</code>	0.431684	0.379889
<code>linear_ramp</code>	<code>default</code>	0.467094	0.321178
<code>step_sigma_theta</code>	<code>default</code>	0.458834	0.328883
<code>periodic_drift</code>	<code>default</code>	0.439352	0.367232

Table 6: Best statistical-calibration variant per scenario within the bounded extension lane. Gap is the hybrid residual LER minus the best statistical calibration LER; this table does not remove the accepted no-promotion gate.